

3 a.m. Blast Radius: Facilitator Guide

For the people delivering the session. It contains spoilers, including the easter egg. Attendees get [LAB-GUIDE.pdf](#); you get this, plus the attendee guide, which you should know cold.

03:07. profile-cache p99 latency 4.2s, hit rate 31% and falling. Declared tier 3. It can probably wait until morning. **It can't. The graph is how you find out why, and what to do about it.**

Printed copy. The living version is FACILITATOR-GUIDE.md ; it is generated, so edit build/docs/FACILITATOR.tmpl.md , not this file.

1. The session on one page

Audience	About 60 JPMorgan Chase engineers, Columbus. Mixed Cypher experience; assume most have none
Slot	60 minutes: a 45-minute lab inside it
Where it runs	Each attendee's own database, in Neo4j Browser on their corporate laptop. Neo4j 5.26, APOC available, no GDS, no Bloom. Nothing to install
What they do	Copy, paste and read 17 queries, then change the graph themselves (Part 4) and watch the answer move. 11 optional stretch challenges
The story	A fictional bank. A "tier 3" cache pages you at 3 a.m. Through one link nobody ever classified, it sits under up to 26 of 30 customer journeys
The graph	1,094 nodes, 2,460 relationships. Fictional throughout

Three things the room should leave with. Say them at the start and again at the end.

1. **The dependency that matters isn't in the catalog.** It's in the traces, and in the graph.
2. **"We don't know" is measurable.** The graph says how much of the answer is assumption, and which single fact would change it most.
3. **Much of an incident's cost is routing, not fixing.**

The line to leave them with: *"The architecture diagram shows what we intended. The graph shows what we built, and how much of it nobody has checked."*

The clock

The attendee guide counts minutes from the start of the lab. Add the opening and you get the slot clock.

Slot	Lab	What	You
0:00		Welcome, the hook, the rules, everyone runs R0 together	Lead
0:06	0	Part 1: The page. Steps 1 and 2	Lead drives
0:14	8	Part 2: How bad is it? Steps 3 to 5	Lead drives
0:26	20	Part 3: Why can't we tell? Steps 6 to 8	Lead drives
0:36	30	Part 4: Your turn. Steps 9 to 14	Lead drives, floor busy
0:44	38	Part 5: What do we do? Steps 15 and 16	Lead drives
0:49	43	The decision	Lead
0:51	45	Close, questions, where to find everything	Lead
0:58		Buffer	

Those are estimates. Nobody has run this with a room yet (see section 9). The close has seven minutes of give in it for a reason.

Who you need

Role	How many	Does
Lead	1	Drives the big screen, tells the story, watches the clock
Floor helpers	3 (one per 15 to 20 attendees)	Walk the room in zones, fix the problems in section 6, never take over a keyboard
Ops contact	1, from JPMC, in the building	Can reset one database by running the loader (section 5). Without this person, the attendee guide's promise that "we can restore your database in seconds" is not true
Timekeeper	Optional; a helper can do it	Calls the checkpoints in section 7

2. Before you facilitate: learn the story

1. **Do the lab yourself, start to finish, as an attendee.** 45 minutes. Use a rehearsal database (section 10), not the demo one, because Part 4 writes.
2. **Read STORYLINE.md** (spoilers). It has the six scenes, the easter egg and everything that is planted.
3. **Know the numbers in Appendix A without looking.** You will be asked, and the room can tell.
4. **Say out loud, once, the one-paragraph version:** *A cache pages you at 3 a.m. and the page says tier 3. 19 alerts fire from 11 teams; the graph says one is the cause. It sits under 48 services and, in the worst case, 26 of 30 customer journeys, through one link nobody classified. You change the graph to find out what each answer would mean, and then decide who to wake.*

3. What must be true before the day

Still to confirm with JPMC

None of this is confirmed yet. Everything we tested assumes the "yes" answer. The right-hand column is what to do if it's "no".

Question	Why it matters	If "no"
Does each attendee's login open on their own database ?	Every query runs without naming one. If it opens <code>neo4j</code> or another database, every query returns nothing	Browser lets the user pick a database in the top bar; each attendee would do that once. Slower, and error-prone with 60 people
Who loads the 60 databases, and how?	<code>graph/load.cypher</code> is pure Cypher: 327 kB, 40 statements, no files, no plugins. It runs in seconds per database	If only a Cypher console is available, the batches can be pasted statement by statement. Painful; avoid
Who can re-run the loader for one database during the session?	It is the only real reset (section 5)	Without it, a broken database stays broken. Part 4's own reset (Step 14) still works
Do attendees have schema privileges in their database?	The loader begins by creating constraints and indexes. Only matters if attendees re-run it themselves, which we don't plan	Not needed otherwise. We tested a non-admin user with <code>GRANT ALL</code> on their own database, which works
Query time limits and server load with 60 people at once	Every query finished in well under a second on a quiet test server. JPMC's shared server is unmeasured	Ask what the transaction timeout is. If someone sets it below a few seconds, tell us
Can the pre-event email carry a repo link or zip , and do they get through security filtering?	Attendees need <code>LAB-GUIDE.md</code> or the zip to copy queries from	Print the PDF and read from it. Do not copy queries out of the PDF ; see section 6
Which Browser version is deployed, and can people paste into it?	We tested a current Browser. Older ones look different	Screenshots in the guide won't match; the text still will
Will the room network reach the Browser URL?	Same laptop and network as the readiness check	Run the readiness check from the room the day before, if possible

The sandbox smoke test (the single most useful thing JPMC can do)

We tested on a clean Neo4j 5.26 laid out like the sandbox. **We have not run on the sandbox itself.** Ask one JPMC person to do this with a **non-admin test account**, a day or more ahead, and send screenshots:

1. Log in to Neo4j Browser. Run `R0`. It should show **232 components** and **228 incidents**.
2. Run `B1b`. Expect 19 rows.
3. Run `H1`, `H2`, `H3`, `H6`, `H7` (Part 4). Expect 26 and 4 at `H1` and `H7`; 9 and 4 at `H3`.

That is five minutes, and it exercises login, home database, read access, write access, and the lab's slowest queries.

Countdown

When	Do
3 weeks	Send JPMC the questions above. Agree who is the ops contact
2 weeks	JPMC loads one pilot database; the smoke test runs; we fix whatever it finds
1 week	JPMC loads all databases. Send the pre-event email (Appendix C). Run <code>make verify</code> once more; it must pass
2 days	Count readiness check-ins (Appendix B). Chase anyone at zero
1 day	Rehearse with the team (section 10). Reset the presenter's database. Re-send to anyone still not ready
Morning of	Presenter logs in on the room's network and projector. Run <code>R0</code> . Run <code>H1 : 26 and 4</code> . If not, run <code>H6</code>
Before doors	Open <code>LAB-GUIDE.pdf</code> on the presenter's laptop as the fallback. Browser zoom to 125% or more. Collapse the left sidebar

The presenter's laptop

- **Run the same steps in your own database**, from the same guide, with the same paste. Typing live invites typos.
- **Keep a local copy of the whole lab as a fallback:** `make up` and `make build` give you the same graph on Docker, with no network. Not a substitute; a safety net.
- **If Browser dies on the big screen**, present from `LAB-GUIDE.pdf`. It has every query's real result.

4. Run of show

Each step: **do** what is on the screen, **say** the idea (in your own words), **land** the number, **watch for** the trap. Query ids match `queries/*.cypher`. "Predict" prompts are in the attendee guide: ask for a guess out loud or by show of hands, *then* run.

Opening (slot 0:00 to 0:06)

1. **Say, in the first minute:** *"Everything here is fictional: the bank, the teams, the incidents, the numbers. Nothing describes any real institution's architecture."*
2. **The hook.** Read the page aloud: *profile-cache, p99 latency 4.2 seconds, hit rate 31% and falling. Declared tier 3, which this bank's own rating says can wait until morning.* **Ask the room: "Do you get out of bed?"** Take a show of hands. Leave it hanging.
3. **The rules, in a breath.** You run every step on the big screen at the same time as they do. If it doesn't work on your laptop, keep watching. Copy the whole block. One query at a time. Part 4 is the only part that changes anything, and only in *your* database.
4. **Everyone runs R0 together.** Paste it on screen. They should see their own database name, **232** components and **228** incidents. *Hands up if you see a number.* Anyone at zero, or with an error: a helper goes to them **now**, not later.

Part 1: The page (lab 0 to 8)

Step 1 • B1 (about 2 minutes)

- **Do:** Run it.
- **Say:** *"One alert. A Redis cache. Declared tier 3, owned by Data Platform. If you'd been paged, you'd go back to sleep."*
- **Land:** One row. `profile-cache`, declared tier 3.
- **Watch for:** If someone says they'd roll over, good. Let it stand.

Step 2 • B1b (about 5 minutes)

- **Predict first:** *"Of 19 alerts across 11 teams in 7 minutes, how many are the actual cause?"* Take guesses out loud.
- **Do:** Run it. Point at the two count columns: `alerting_dependents` and `alerting_dependencies`.
- **Say:** *"For each alerting component, how many other alerting components depend on it, and how many does it depend on? Lots depend on it and it depends on nothing alerting: cause. Depends on something alerting: symptom. Neither way: nothing to do with the rest."*
- **Land:** **1 likely cause** (`profile-cache`, 9 alerting dependents), **10 symptoms, 8 unrelated.**
- **Make the point:** The cause is the quietest, lowest-tier component in the room. Two symptoms nobody would link to a cache: `campaign-svc` and `agent-desktop-svc`. *"Who is paging Marketing Tech right now?"*
- **Watch for:** `wire-svc` appears twice: a duplicate alert, real alerting has them. "Unrelated" means *unrelated to the other alerts through any dependency*, not "fine". Those teams may have their own problem. Say so if asked.

Part 2: How bad is it? (lab 8 to 20)

Step 3 • B2 (about 3 minutes)

- **Do:** Run it. **Don't read the examples column aloud.**
- **Say:** *"Everything that fails if the cache goes, in rings by how many hops away. Ring one is a single service: the one that talks to the cache. By ring three you are at wires, transfers, payments."*
- **Land: 4 rings** of 1, 12, 27, 8 services: **48 in all**. This is the worst case: it follows every dependency not known to be soft.

Step 4 • B2b (about 4 minutes) • the first pivot

- **Predict first:** *"Of those 48, how many can you prove will fail?"* Take a few guesses before you run it.
- **Do:** Run it.
- **Land: 14 proven, 34 assumed, 28 degrade only.**
- **Say:** *"Proven means every link on the path is a known hard dependency. Assumed means a link nobody ever classified is holding it on the list. That is 71 percent of the failing list resting on a guess. Hold that thought."*
- **Watch for:** The distinction between **confirmed** (provable) and **hard** (assume the worst) is the spine of the lab. If the room is lost, stop and say it once more.

Step 5 • B3b (about 4 minutes)

- **Do:** Run it.
- **Land: A range: 473 to 32,040** attempts an hour at 03:00, about **68** times apart. By 07:00, **6,380 to 319,720**: about **10** times the 03:00 figure.
- **Say:** *"If the incident commander asks how many customers are affected, an honest answer is a range of 68 to one, because of what nobody has checked."*
- **Watch for: "Attempts" are not customers.** Someone who logs in and then checks a balance is two attempts. A journey is a *thing customers do*. Say it every time someone asks.

Part 3: Why can't we tell? (lab 20 to 30)

Step 6 • B2c (about 3 minutes)

- **Do:** Run it.
- **Say:** *"These are the links on the path to the cache that nobody ever classified, ranked by how much sits above each."*
- **Land:** Top row: `entitlements-svc -> customer-profile-svc`, **33 services** and **26 journeys** above it. The next has 5 journeys. `seen_via` is `observed`: seen in traces, **never declared in the catalog**.
- **Make the point:** One link carries most of the doubt. Check that one link and most of the range collapses.

Step 7 • B6b (about 2 minutes)

- **Do:** Run it.
- **Land:** `CHG-1873`, 2025-06-03, **15 months ago**, by Identity & Access, **no risk review**.
- **Say:** *"Nobody did anything wrong. A team resolved group membership through the profile service. A perfectly normal change. It just never made it into the catalog."*
- **Watch for:** Don't let it turn into blame. The point is the process gap, not the team.

Step 8 • B6c : the replay (about 5 minutes) • the payoff of Part 3

- **Predict first:** "How many journeys depended on the cache the day before that change?"
- **Do:** Run it.
- **Land:** 9 journeys to 26, 22 services to 48, about 5 times the daily attempts at risk.
- **Say:** "One undeclared call, 15 months ago, turned a tier-3 cache into part of the foundation of login."
- **If asked how:** every dependency carries a `first_seen` date. The query asks the same question twice, keeping only the dependencies that existed by each date. It uses *today's* classifications, so it shows what the structure was, not what anyone knew then.
- **Pause here.** Then: "So far you have only read the graph. Now you decide."

Part 4: Your turn (lab 30 to 38) • the only part that writes

This is the reason the lab is hands-on. **Slow down; do it in order; keep the floor moving.** Details in section 5.

Step 9 • H1: "Write these two numbers down." 26 worst case, 4 proven.

Step 10 • H2: "Suppose entitlements-svc survives a slow profile service. You are declaring the link soft." The result is one row: `links_classified` 1, `flags_refreshed` 617.

Step 11 • H3 (Predict first: "What's the worst case now?")

- **Land:** 9 journeys in the worst case; proven still 4.
- **Say:** "That is exactly the number from the day before the change. Tonight looks like before CHG-1873."

Step 12 • H4: "Now the opposite. It fails when the profile service is slow. Hard."

Step 13 • H5 (Predict first: "How many are now proven?")

- **Land:** 26 worst case, 26 proven.
- **Say:** "What was doubt is now fact. One link. Two answers. And nobody on the call knows which is true."
Show of hands: soft or hard? If the room splits, that split *is* the finding. If it doesn't, ask what they would need to see to be sure.

Step 14 • H6 then H7: "Put it back." H7 must show 26 and 4, the same as H1. **Announce it as a gate:** "Don't move on until H7 matches H1." If someone stops with the link left **soft**, Step 15 shows different changes in the radius from everyone else's, and so do several stretch challenges. (Left **hard**, Step 15 is unaffected, but some stretch challenges still differ.) Step 16 never changes.

Part 5: What do we do? (lab 38 to 43)

Step 15 • B8 (about 3 minutes)

- **Do:** Run it.
- **Land:** 12 changes in 24 hours; 4 are in the blast radius. The closest: CHG-2301, a cache TTL cut from 24 hours to 1 hour, deployed at 22:15, **no risk review**.
- **Say:** "The most likely trigger, not a proven one." A shorter TTL means entries expire sooner, so the hit rate falls and the database behind the cache takes the load. It's a hypothesis, and the graph tells you to roll it back first because it is the closest and the one you can undo.

Step 16 • B7b (about 2 minutes)

- **Do:** Run it.
- **Land:** 211 incidents with a recorded root cause. Routed to the owner: **50 minutes** average. Routed elsewhere: **169**, bouncing between teams **2.1** times.
- **Say, every time:** *"These numbers are invented for the exercise, and the size of the gap is built into the data. The direction is the point."* (See section 9.)

The decision (lab 43 to 45)

Say these four, slowly. They're on the last page of the attendee guide.

1. **Data Platform: roll back** CHG-2301. Closest change, and the one you can undo.
2. **Identity & Access: answer one question.** *Does entitlements-svc survive a slow profile service?* It moves the worst case from 26 journeys to 9.
3. **Tell the other teams in the blast radius.**
4. **Monday:** classify the link, declare it, give the cache a runbook and an honest tier, route incidents to the team that owns the cause.

Close (slot 0:51 to 0:58)

1. **Repeat the three takeaways.**
2. **Give them homework for their own estate.** Three questions to ask Monday morning: - *Which dependencies in our tracing data are not in our catalog?* - *Which of those have anyone ever decided are hard or soft?* - *What is the lowest tier of anything that a top-tier service depends on?*
3. **Where everything is:** the repo or zip from the pre-event email has the guide, every query and the graph. `graph/load.cypher` rebuilds the whole lab on any Neo4j 5.26 or later.
4. **Questions.** Section 8 has the ones you'll get.
5. **If someone found the easter egg, hand them the room** (section 7). If nobody did, don't tell them there is one: send the stretch challenges with the repo and let them discover it.

5. Part 4 in detail

The risk: it's the one place attendees can leave their database in a state that makes later answers wrong.

What can go wrong, and the fix:

Symptom	Cause	Fix
H3 shows 26, not 9	They skipped H2	Run H2 , then H3 again
H5 shows 26 and 4	They skipped H4	Run H4 , then H5
Step 15 lists different changes in the radius, or stretch challenges differ	They never ran Step 14, usually with the link left soft	Run H6 , then H7 . It must read 26 and 4
Everything is stranger than that	They edited a query, or ran something else	The full reset below

Every Part 4 statement is safe to repeat. H2 , H4 and H6 each set the link to a fixed value and recompute every flag, so running one twice changes nothing. H6 **is the universal "back to the start" for this part.**

Full reset of one database (the ops contact does this, or you with their credentials):

```
cypher-shell -a <uri> -u <user> -p <password> -d <database> -f graph/load.cypher
```

It takes a few seconds and ends by printing the node and relationship totals: **1,094** nodes and **2,460** relationships, **plus one node for each time the readiness check was run** (each run adds a small `:Checkin` marker).

We tested it. `build/verify.py` does exactly this as a non-admin user with full rights over their own database, in two cases: after leaving the link half-edited, and after deleting everything. Both come back to the identical graph.

One limit: the loader restores what the lab put there. It does not remove things an attendee *added*. If someone has created their own nodes, wipe first:

```
MATCH (n) DETACH DELETE n;
```

then run the loader. (That also removes their readiness marker; ask them to re-run `R0` if you're counting.)

6. The floor: what helpers need to know

Zones. Split the room into thirds. Walk, don't sit. **Never take over a keyboard:** point, and let them paste.

The first aid table. Most problems are one of these.

What you see	What it is	Fix
No rows; <code>R0</code> shows 0 components	Wrong database	Don't try to fix it. Ops contact, with their username
Error saying something "is not allowed" on <code>neo4j</code> ; "Database not found" ; access denied	Login opened the wrong database	Same
Syntax error	Part of the query was missed when copying	Copy the whole block again, first line to the <code>;</code>
They typed it and it fails	Typos	Paste instead. Queries are written to be pasted
Pasted from the PDF and it fails	Copying out of a PDF can lose line breaks, and we haven't tested what else it changes	Use <code>LAB-GUIDE.md</code> or <code>queries/*.cypher</code> from the repo or zip. Lab queries carry no <code>//</code> comments, which helps, but it is still the unreliable route
A query keeps spinning	Usually an edited query with a long path, or the server is busy	Stop button on the frame. Lab queries take well under a second
Table is tiny, columns clipped	Browser window or zoom	Collapse the left sidebar; zoom out
A tour or tooltip pops up over the editor	Newer Browser versions show a welcome tour	Dismiss it. Ignore the <code>:welcome</code> frame
Several result frames piled up	They're running without clearing	<code>:clear</code>
Numbers differ after Part 4	Step 14 never ran	H6 then H7
They ran <code>R0</code> twice	Harmless	Each run adds a <code>:Checkin</code> marker. Nothing is wrong
"Can I use Bloom ?"		Not for this lab. Not tested

Escalate to the lead, not the ops contact, if the same problem appears for five or more people at once. That's the server or the instructions, not the individual.

7. Pace: checkpoints, cuts and fast finishers

Checkpoints. Say them out loud; the room will pace itself.

By lab minute	You should be at	If behind
8	Finishing Step 2	Skip Step 3 (B2): Step 4 shows the same radius, split by proof
20	Finishing Step 5	Start Part 3 at once. Take the time from the close, not from Part 3
30	Starting Step 9	Protect Part 4. It is the point of the lab. Cut Part 5 if you must
38	Starting Step 15	Run Step 15 only; mention routing in words
43	The decision	Skip Step 16 if you haven't run it, not the decision

Cut order, if you must: Step 3 (early), then Step 16, then Step 15 (late). **Never cut:** Steps 1, 2, 4, 6, 7, 8, or any of Part 4.

Waiting for the room. Ask "who has the result?" and **wait for most hands, not all.** Helpers go to the rest. If a quarter of the room is still stuck after two minutes, ask them to *watch* and keep going. The attendee guide told them they won't miss the point.

If the room is flying. Point them at the stretch challenges. For people who know Cypher, S5 (the datastore mismatches) and S6 (the chain, link by link) are the richest. The answers are in the guide, under each challenge.

The easter egg

Do not mention it. It's the second hotspot, and a reward for curiosity. A :Certificate label sits in the database list and in no part of the lab. **Stretch challenge S11** has two hints.

If someone finds it, give them the room for a minute: "`wildcard.internal.bank expires in 8 days and does not renew itself. Its owner, Shared Services, was disbanded. It's used by 38 services across 16 teams, touching 27 of the 30 journeys. It was last rotated by hand the morning it expired, and took the estate down for 4 hours. A different kind of hidden shared dependency: a 8-day fuse instead of a 3 a.m. page.`" The other 4 certificates on the list are decoys: they expire soon but renew themselves, or matter little.

Two traps:

- "Days" are counted from the **story's** date (2026-09-30), not today's. If someone notices the calendar date doesn't match your session date, that is why.
- The label list (B11a , hint 1) also shows Checkin : the readiness markers. If someone asks, that's the check-in from the first minute.

8. Questions you'll get

"Is this real data? Is this Chase's architecture?" No. Everything is invented: the services, teams, incidents and customer numbers. Say it early; say it again if asked.

"We have a CMDB / a service catalog. We have all this." You may. The edge that mattered here was *in the traces and nowhere else*. A graph is where the catalog, the traces, the change records, the incident tickets and the org chart meet, and the hard part in real life is getting and reconciling those sources.

"How would we get this data?" Typically from the service catalog or CMDB, distributed traces, the deployment and change system, incident tickets, and team and on-call data. The work is mapping names across them: real data is worse than this lab's, with sources that disagree, edges missing and names that don't match.

"Does this scale?" This graph is a lab, not a bank: 1,094 nodes, and every lab query finishes in well under a second. **Don't quote a benchmark; the lab doesn't prove one.** Neo4j runs graphs far larger than this, but that is a general statement, and not something this lab demonstrates.

"Why not a SQL recursive query?" You can write the traversal. The graph's advantage shows when you add the next question: owners, changes, runbooks, history. Each is another hop, not another join design.

"Is confirmed / hard built into Neo4j?" No. Those are two flags this lab *derives* from `critical` and `active` on each dependency. The modelling choice is the point: *unknown* is not *soft*, so the graph keeps both a provable best case and a worst case you must plan for.

"Where did PageRank come from? Can we run GDS?" Computed once with Graph Data Science on our side, then shipped as an ordinary property, because the sandbox has no GDS. Only S10 uses it; it's an independent check that doesn't rely on anything declared.

"Is the routing penalty real?" No. The direction is plausible; the size is built into the generated data. Say so before they ask.

"Can an LLM do this in plain English?" Possible, but it isn't part of this lab and we haven't built or tested it here. If it came up, the thing worth keeping explicit in any such tool is the difference between *proven* and *assumed*.

"Why did Tap to pay survive?" Its link to the cache is soft, with a fallback. It's the one place someone designed for this.

"Why do two alerts name `wire-svc`?" A duplicate alert, left in on purpose: real alert storms have them.

"Can I run this myself?" Yes. The repo has `graph/load.cypher`, which rebuilds the whole lab on any Neo4j 5.26 or later in a few seconds.

9. Be honest about

What is planted. The graph is generated. Deliberately planted: the unclassified link and its history; the 6 cache incidents, all misrouted, with outages at low hit rates; the **routing penalty**; tonight's alert storm and changes; and the easter egg. The rest, about 228 incidents and 455 changes, is seeded background. **Say so before the room asks.**

What has and has not been tested.

	Status
Every query and the loader on a clean Neo4j 5.26 , as a non-admin user with only their own database	Tested , every build (<code>make verify</code>). Also on 2026.08 with identical results
The loader run twice, and after an edited or an emptied database	Tested
Neo4j Browser : login to the home database, <code>R0</code> , <code>B1b</code>	Tried once , on a current Browser against the dev stack
The other steps through Browser	Not tried . They ran through <code>cypher-shell</code> as the same kind of user
JPMC's sandbox , their Browser version, their network and server load	Not tested . See section 3
The timings in this guide	Estimates . No room has run it. Your rehearsal is the first data
Bloom / Explore	Not used and not tested

What the lab cannot say. It does not say real estates look like this, or how a bank should build things, or that Neo4j beats a service catalog. It shows what a connected question looks like.

10. Rehearsal

Part 4 writes to the graph, so a team can't rehearse in a single shared database. This gives each person their own, laid out like the sandbox:

```
make up           # the dev stack, if it isn't running
make build        # load the graph
make rehearsal N=4 # lab-user01 to lab-user04, each with its own login
make rehearsal-down N=4 # drop them again
```

Each person opens Neo4j Browser at the address `make urls` prints, and logs in as `userNN` with the password the command prints. That user is **not an admin** and its home database is `lab-userNN`: the same shape as an attendee. These are throwaway logins for the local stack only.

Run it in real time: one person leads, the others follow as attendees on their own databases, a fifth person keeps the clock. **Write down:**

Per step	Minutes it really took, with copy and paste included
Where the lead lost the room	The step, and what was unclear
Where anyone got stuck	And what fixed it
Anything the guide says that wasn't true	

Then change the **templates** (`build/lab-guide.template.md`, `build/docs/FACILITATOR.tpl.md`), never the generated files, and run `make docs`.

Appendix A: the numbers

Know these. Everything is computed from the tested results, so it's right for this build.

The graph	1,094 nodes, 2,460 relationships: 175 services, 57 datastores, 25 teams, 30 journeys, 228 incidents, 455 changes, 71 runbooks, 34 certificates
R0	232 components, 228 incidents
Step 2	19 alerts, 11 teams, 7 minutes: 1 cause (profile-cache), 10 symptoms, 8 unrelated
Step 3	4 rings: 1, 12, 27, 8. 48 services in the worst case
Step 4	14 proven, 34 assumed (71%), 28 degrade
Step 5	473 to 32,040 attempts/hour at 03:00 (×68); 6,380 to 319,720 at 07:00 (×10)
Step 6	entitlements-svc → customer-profile-svc : 33 services, 26 journeys. Next: 5
Step 7	CHG-1873 , 2025-06-03, 15 months, no risk review
Step 8	9 → 26 journeys ; 22 → 48 services; 1,352,000 → 6,704,000 attempts/day (×5)
Part 4	H1 : 26 / 4 → soft H3 : 9 / 4 → hard H5 : 26 / 26 → restored H7 : 26 / 4 (worst case / proven)
Step 15	12 changes, 4 in the radius. CHG-2301 , 22:15, no review
Step 16	50 min to the owner (149 incidents); 169 min elsewhere (62), 2.1 reassignments
S5	4 mismatches; profile-cache first: declared tier 3, 26 journeys worst case
S7	6 cache incidents, 4 teams, 0 to the owner, 1,325 minutes
S10	PageRank: customer-profile-svc first, then ledger-svc
S11	wildcard.internal.bank : 8 days, no renewal, owner disbanded, 38 services, 16 teams, 27 journeys

Appendix B: operating commands

Count readiness check-ins (the ops contact runs this with admin rights, from any machine with cypher-shell; adjust the address, user and database names to whatever JPMC uses):

```
for db in $(printf 'lab-user%02d ' $(seq 1 60)); do
  n=$(cypher-shell -a <uri> -u <admin> -p <password> -d "$db" --format plain \
    "MATCH (c:Checkin) RETURN count(c)" 2>/dev/null | tail -1)
  echo "$db  ${n:-NO DATABASE OR NO ACCESS}"
done
```

0 means the database exists and the person hasn't run the check yet. **A missing value** means no such database, or no access. (We tested this loop's mechanics on a throwaway server; we haven't run it against JPMC's naming.)

Reset one database: section 5.

Where things are:

The lab queries, in order	<code>queries/demo-queries.cypher</code> , <code>queries/hands-on.cypher</code> , <code>queries/readiness.cypher</code>
The whole graph	<code>graph/load.cypher</code>
Prove it still works	<code>make verify</code> (about 90 seconds; needs Docker)
Regenerate the documents	<code>make docs</code>
Rehearsal databases	<code>make rehearsal N=4</code> , <code>make rehearsal-down N=4</code>

Appendix C: the pre-event email (draft for JPMC to send)

Subject: Graph lab on : please do a two-minute check beforehand

You'll work in your own Neo4j database in your web browser. There is nothing to install and no Neo4j experience is needed.

Before the session (by): 1. Open and log in: username `<...>`, password `<...>`. 2. Open the lab guide (), find "**The readiness check**" and paste the one query into the editor. Press Ctrl+Enter (Windows) or Cmd+Enter (Mac). 3. You should see one row with your database name and **232 components**. If you see **0**, an error, or can't log in, **don't try to fix it**: reply to with a screenshot.

Bring: your corporate laptop, charged, on the network you tested from. Everything in the lab is fictional.